

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 10 | Stacks: LIFO, Implementations and Applications

Week 10 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 10 at a glance

Item	Details
Lab title	Stacks: LIFO, Implementations and Applications
Week	10 of 16
Prerequisites	Lab 03 (Arrays), Lab 08 (Linked Lists)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab10_Instructor.ipynb , Lab10_Student.ipynb
Next lab	Lab 11: Queues, Circular Queues and Deques

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain the **stack ADT** and the **LIFO** principle with real examples.
2. Implement the stack operations **push**, **pop**, **peek/top**, **size**, **is_empty** in **O(1)**.
3. Implement a stack using an **array (Python list)**, a **fixed-size array** (with overflow/underflow) and a **linked list**.
4. Check **balanced brackets** using a stack.
5. Convert **infix to postfix** and **evaluate postfix** expressions.
6. Use a **monotonic stack** to solve “next greater element” problems in **O(n)**.
7. Explain how the **function call stack** and **undo/redo** work.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain stack as an abstract data type	Understand (C2)
CLO-2	Implement stacks and stack-based algorithms	Apply (C3)
CLO-3	Apply stacks to solve problems	Apply / Analyse (C3–C4)

2. Motivation: The Most Recent Thing First

Many problems share one pattern: **the last thing that came in is the first thing that must go out**. When you press Ctrl+Z, the **last** change is undone first. When a function finishes, control returns to the function that called it **most recently**. A **stack** is the data structure for this pattern.

Analogy: a stack of plates

At a wedding buffet, clean plates are stacked. The waiter puts a new plate **on top**, and guests take a plate **from the top**. Nobody pulls a plate from the middle. Last In, First Out: **LIFO**.

2.1 Stacks in Daily Life

Example	Push	Pop
Undo in MS Word / Photoshop	Every edit you make	Ctrl+Z removes the latest edit
Browser Back button	Every page you visit	Back goes to the most recent page
Pile of books / exam copies on a table	Put one on top	Take from the top
Bangles on a wrist	Last bangle put on	Is the first to come off
Chips in a Pringles can	Filled from the top	Eaten from the top

2.2 Stacks in Industry

- **Every running program** uses a call stack to manage function calls and local variables. A “stack overflow” error (and the famous website) is named after it.
- **Compilers and interpreters** use stacks to check syntax (matching brackets, HTML/XML tags) and to evaluate expressions.
- **Calculators** convert your expression to postfix and evaluate it with a stack.
- **Code editors (VS Code)** highlight matching brackets and detect missing ones using a stack.
- **Depth-First Search** in graphs (Lab 16) and backtracking (maze solving) use stacks.

2.3 Stacks in Research

- **Programming languages and compilers** research: parsing (LL/LR parsers), virtual machines (the Python and Java VMs are stack machines).
- **NLP**: classic dependency parsers (shift-reduce parsers) use a stack and a buffer to build sentence trees.
- **Computational geometry**: the Graham scan algorithm finds the convex hull of points with a stack.

2.4 Why It Matters in 2026–2030



A small structure with a big footprint

Stacks appear in every layer of computing: hardware, operating systems, compilers, browsers, AI agents that plan and backtrack. Monotonic stacks are now a standard interview topic. Mastering the stack also prepares you to understand recursion deeply, because **every recursive algorithm can be rewritten with an explicit stack**.

3. Part A: The Stack ADT

An **Abstract Data Type (ADT)** describes **what operations** a structure supports, not **how** it is implemented. The stack ADT has these operations:

Operation	Meaning	Time
<code>push(x)</code>	Add x on the top	O(1)
<code>pop()</code>	Remove and return the top element	O(1)
<code>peek() / top()</code>	Return the top element without removing it	O(1)
<code>is_empty()</code>	Is the stack empty?	O(1)
<code>size()</code>	Number of elements	O(1)

Error	When
Stack underflow	<code>pop()</code> or <code>peek()</code> on an empty stack
Stack overflow	<code>push()</code> on a full fixed-size stack

Stack operations step by step

```

push(10)      push(20)      push(30)      pop() -> 30    peek() -> 20
| 10 | <-top  | 20 | <-top  | 30 | <-top  | 20 | <-top  | 20 | <-top
+-----+    +-----+    +-----+    +-----+    +-----+

```



4. Part B: Implementations

4.1 1. Using a Python List (Dynamic Array)

The **end** of a Python list is the top of the stack: **append** and **pop()** at the end are $O(1)$ amortised. Never use the front (**insert(0, x)**, **pop(0)**), which is $O(n)$.

```
class Stack:
    def __init__(self):
        self._items = []

    def push(self, x):
        self._items.append(x)

    def pop(self):
        if self.is_empty():
            raise IndexError("stack underflow")
        return self._items.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("stack is empty")
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0

    def size(self):
        return len(self._items)

    def __str__(self):
        return "bottom " + str(self._items) + " top"

s = Stack()
for x in [10, 20, 30]:
    s.push(x)
print(s, "| pop:", s.pop(), "| peek:", s.peek(), "| size:", s.size())
```

4.2 2. Fixed-Size Array Stack (Overflow and Underflow)

In C and embedded systems, the stack has a fixed capacity. We keep an index **top** (-1 when empty).

```
class ArrayStack:
    def __init__(self, capacity):
        self.data = [None] * capacity
        self.top = -1

    def push(self, x):
        if self.top == len(self.data) - 1:
            raise OverflowError("stack overflow")
        self.top += 1
        self.data[self.top] = x

    def pop(self):
        if self.top == -1:
            raise IndexError("stack underflow")
        x = self.data[self.top]
        self.data[self.top] = None
        self.top -= 1
        return x

a = ArrayStack(2)
a.push("A"); a.push("B")
try:
    a.push("C")
except OverflowError as e:
    print("Error:", e)
print(a.pop(), a.pop())
```

4.3 3. Linked-List Stack

The **head** of a singly linked list is the top: **push_front** and **pop_front** are $O(1)$ and the stack never needs resizing.

```

class LinkedStack:
    class _Node:
        def __init__(self, data, next):
            self.data, self.next = data, next

    def __init__(self):
        self.head = None
        self.count = 0

    def push(self, x):
        self.head = self._Node(x, self.head)
        self.count += 1

    def pop(self):
        if self.head is None:
            raise IndexError("stack underflow")
        x = self.head.data
        self.head = self.head.next
        self.count -= 1
        return x

    def peek(self):
        return None if self.head is None else self.head.data

ls = LinkedStack()
for ch in "DSA":
    ls.push(ch)
print(ls.pop() + ls.pop() + ls.pop())    # reversed: ASD

```

Implementation	push/pop	Pros	Cons
Python list	$O(1)$ amortised	Simplest, fast, cache-friendly	Occasional resize
Fixed array	$O(1)$	No resizing, predictable memory	Overflow when full
Linked list	$O(1)$	Never full, no resize	Extra memory per node

5. Part C: Classic Applications

5.1 Balanced Brackets

Scan the expression. Push every **opening** bracket. For a **closing** bracket, the top of the stack must be the matching opening bracket; pop it. At the end, the stack must be empty.

```
def is_balanced(expr):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in expr:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

for e in ["{[a + b] * (c - d)}", "(a + b)", "((x)", "print(arr[i])"]:
```

```
print(f"{e:<22} -> {is_balanced(e)}")
```

5.2 Infix, Prefix and Postfix

Notation	Example	Used by
Infix	A + B * C	Humans; needs precedence rules and brackets
Prefix (Polish)	+ A * B C	Some compilers, LISP
Postfix (Reverse Polish)	A B C * +	Calculators, stack machines; no brackets needed

5.3 Evaluating a Postfix Expression

Read tokens left to right. **Number** → push. **Operator** → pop two numbers (the first pop is the **right** operand), apply the operator, push the result. The final value on the stack is the answer.

Token	Action	Stack
5	push	[5]
3	push	[5, 3]
2	push	[5, 3, 2]
*	$3 \times 2 = 6$	[5, 6]

Token	Action	Stack
+	$5 + 6 = 11$	[11]
4	push	[11, 4]
-	$11 - 4 = 7$	[7]

```
def eval_postfix(tokens):
    stack = []
    for t in tokens.split():
        if t in "+-*/":
            right = stack.pop()
            left = stack.pop()
            if t == "+": stack.append(left + right)
            elif t == "-": stack.append(left - right)
            elif t == "*": stack.append(left * right)
            else: stack.append(left / right)
        else:
            stack.append(float(t))
    return stack.pop()

print(eval_postfix("5 3 2 * + 4 -"))      # 5 + 3*2 - 4 = 7
```

5.4 Infix to Postfix (Shunting-Yard Algorithm)

Rules: operands go straight to the output. (is pushed.) pops operators to the output until (. For an operator, first pop operators of **higher or equal precedence** (for left-associative operators), then push it. At the end, pop everything.

```

PREC = {"+": 1, "-": 1, "*": 2, "/": 2, "^": 3}

def infix_to_postfix(expr):
    out, stack = [], []
    for t in expr.split():
        if t == "(":
            stack.append(t)
        elif t == ")":
            while stack[-1] != "(":
                out.append(stack.pop())
            stack.pop() # discard "("
        elif t in PREC:
            while (stack and stack[-1] != "(" and
                   (PREC[stack[-1]] > PREC[t] or (PREC[stack[-1]] == PREC[t] and t
            != "^"))):
                out.append(stack.pop())
            stack.append(t)
        else:
            out.append(t) # operand
    while stack:
        out.append(stack.pop())
    return " ".join(out)

print(infix_to_postfix("A + B * C"))
print(infix_to_postfix("( A + B ) * C - D / E"))
print(infix_to_postfix("2 ^ 3 ^ 2")) # ^ is right-associative

```



6. Part D: Monotonic Stack

A **monotonic stack** keeps its elements in increasing or decreasing order. It solves “**next greater / smaller element**” problems in **O(n)** instead of $O(n^2)$, because each element is pushed and popped at most once.

Problem: for each day’s share price, find the next day with a **higher** price (–1 if none).

```

def next_greater(prices):
    result = [-1] * len(prices)
    stack = [] # indexes of days still waiting for a
    higher price
    for i, p in enumerate(prices):
        while stack and prices[stack[-1]] < p:
            result[stack.pop()] = p # today is the answer for that earlier
            day
        stack.append(i)
    return result

prices = [100, 80, 120, 90, 95, 130]
print(next_greater(prices))

```

7. Part E: The Call Stack and Recursion

Python itself uses a stack to run functions. Any recursive function can be rewritten using your own stack, which avoids Python's recursion limit.

```
def factorial_with_stack(n):
    stack = []
    while n > 0:
        stack.append(n)
        n -= 1
    result = 1
    while stack:
        result *= stack.pop()
    return result

import traceback
def a(): b()
def b(): print("".join(traceback.format_stack(limit=3)[-1]))
print(factorial_with_stack(10))
a() # shows the call stack: a -> b
```

8. C++ Corner

```
C++
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isBalanced(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (st.empty()) return false;
            char t = st.top(); st.pop();
            if ((c == ')') && t != '(') || (c == ']') && t != '[') || (c == '}') && t
            != '{')
                return false;
        }
    }
    return st.empty();
}

int main() {
    stack<int> s; // std::stack: push, pop, top, empty, size
    s.push(10); s.push(20);
    cout << s.top() << endl; // 20
    s.pop(); // note: pop() returns void in C++
    cout << isBalanced("{[()]}") << isBalanced("]") << endl; // 10
}
```

9. Common Mistakes

Mistake	Effect	Fix
Using <code>list.pop(0)</code> / <code>insert(0, x)</code> as the top	O(n) per operation	Use the end of the list
Popping without checking empty	Underflow / IndexError	Check <code>is_empty()</code> first
Wrong operand order in postfix	<code>5 3 -</code> gives -2 instead of 2	First pop = right operand
Forgetting the stack must be empty at the end	<code>((x)</code> reported as balanced	Return <code>not stack</code>
Treating <code>^</code> as left-associative	Wrong postfix for <code>2 ^ 3 ^ 2</code>	Do not pop equal-precedence <code>^</code>

10. Lab Tasks

Solve all tasks in `Lab10_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Reverse Words with a Stack

Write `reverse_words(sentence)` that reverses the **order of words** using a stack (push every word, then pop). Do not use slicing or `reversed()`.

```
"Data Structures are fun" → "fun are Structures Data"
```

Task 2 HTML Tag Checker

Web browsers must check that HTML tags are properly nested. Given a list of tags like `"<p>"` and `"</p>"`, return `True` if every closing tag matches the most recent unclosed opening tag.

Hint: same idea as balanced brackets; a closing tag starts with `</`.

```
["<html>", "<body>", "<p>", "</p>", "</body>", "</html>"] → True
["<b>", "<i>", "</b>", "</i>"] → False
```

Task 3 Min Stack

Design `MinStack` with `push`, `pop`, `top` and `get_min`, all **O(1)**. Hint: keep a second stack that stores the current minimum at each level.

```
push(5), push(2), push(7) → get_min 2; pop() → get_min 2; pop() → get_min 5
```

Task 4 Evaluate Infix Expressions

Combine `infix_to_postfix` and a postfix evaluator to write `calculate(expr)` for space-separated infix expressions with numbers, `+`, `-`, `*`, `/`, `^` and brackets. Return a float.

```
calculate("( 2 + 3 ) * 4 - 6 / 2") → 17.0
```

Task 5 Daily Temperatures

For each day in a list of Lahore temperatures, return how many days you must wait for a **warmer** day (0 if none). Use a monotonic stack for $O(n)$.

```
[31, 33, 29, 30, 35, 28] → [1, 3, 1, 1, 0, 0]
```

Task 6 Text Editor with Undo and Redo

Build **Editor** with `type(text)` (append text), `delete(k)` (remove last `k` characters), `undo()` and `redo()`, and a `text` attribute.

Use **two stacks**: every change is pushed onto the undo stack; undo moves it to the redo stack; any new change clears the redo stack.

```
type("Hello"); type(" World"); delete(6) → "Hello"
undo() → "Hello World"; undo() → "Hello"; redo() → "Hello World"
```

Task 7 Largest Rectangle in a Histogram

A bar chart of daily rainfall has bar heights `h`. Find the area of the **largest rectangle** that fits entirely under the bars. Brute force is $O(n^2)$; use a monotonic increasing stack for **$O(n)$** .

Hint: when a bar is popped, it is the shortest bar of a rectangle that extends from the new stack top + 1 to the current index - 1.

```
[2, 1, 5, 6, 2, 3] → 10      # bars 5 and 6, width 2
```



11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab10** folder.

#	Problem	Level	Concept practised
20	Valid Parentheses	Easy	Balanced brackets
1047	Remove All Adjacent Duplicates In String	Easy	Stack of characters
844	Backspace String Compare	Easy	Stack simulation
155	Min Stack	Medium	Auxiliary stack (Task 3)
150	Evaluate Reverse Polish Notation	Medium	Postfix evaluation
739	Daily Temperatures	Medium	Monotonic stack (Task 5)
496	Next Greater Element I	Easy	Monotonic stack
84	Largest Rectangle in Histogram	Hard	Monotonic stack (Task 7)



12. Home Task

1. Finish all tasks and push `Lab10_Student.ipynb` to GitHub.
2. Write `infix_to_prefix(expr)` (hint: reverse the expression, swap brackets, convert to postfix, reverse the result).
3. Implement **two stacks in one array** (one grows from the left, one from the right) so that no space is wasted.
4. **Mini-project: Scientific Calculator.** Build a calculator that accepts expressions without spaces (e.g., `3+4*(2-1)^2`), tokenises them, converts to postfix, evaluates, and reports errors such as unbalanced brackets or division by zero.
5. Solve at least **3** LeetCode problems from the list.



13. Viva Questions

1. What is LIFO? Give two real-life examples.
2. What is an abstract data type?
3. List the stack operations and their time complexity.
4. What are stack overflow and underflow?
5. Which end of a Python list should be the top of the stack? Why?
6. How do you check balanced brackets using a stack?
7. Convert `(A + B) * C` to postfix.
8. Evaluate the postfix expression `6 2 / 3 +`.
9. What is a monotonic stack and why is it $O(n)$?
10. How does the call stack work during recursion?



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.

15. References and Further Learning

- E. W. Dijkstra's Shunting-yard algorithm (1961); see the Wikipedia article "Shunting yard algorithm".
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.